DAA I

```
def recursive(n):
    if n <= 1:
        return n
    return recursive(n - 1) + recursive(n - 2)



def non_recursive(n):
    a, b = 0, 1
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b



n = int(input("Enter number of terms: "))

print("Recursive Fibonacci:")
for i in range(n):
    print(recursive(i), end=" ")

print("\nNon-Recursive Fibonacci:")
non_recursive(n)
```

DAA II

```
import heapq

def huffman(freq):
    heap = [[f, [ch, ""]] for ch, f in freq.items()]
    heapq.heapify(heap)
```

```
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        for item in left[1:]:
            item[1] = "0" + item[1]

        for item in right[1:]:
            item[1] = "1" + item[1]

        heapq.heappush(heap, [left[0] + right[0]] + left[1:] + right[1:])

    return sorted(heap[0][1:], key=lambda x: x[0])


freq = {'A': 5, 'B': 9, 'C': 12, 'D': 13, 'E': 16, 'F': 45}

codes = huffman(freq)

print("Character  Frequency  Huffman Code")
for ch, code in codes:
    print(ch, "         ", freq[ch], "        ", code)
```

DAA III

```
n = int(input("Enter number of items: "))

weights = []
profits = []
```

```
for i in range(n):
    weights.append(int(input("Enter weight: ")))
    profits.append(int(input("Enter profit: ")))

capacity = int(input("Enter knapsack capacity: "))

items = []
for i in range(n):
    items.append((profits[i] / weights[i], weights[i], profits[i]))

items.sort(reverse=True)

total_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        total_profit += profit
        capacity -= weight
    else:
        total_profit += ratio * capacity
        break

print("Maximum Profit:", total_profit)
```

DAA IV

```
n = int(input("Enter number of items: "))

weights = []
profits = []

for i in range(n):
```

```
    weights.append(int(input("Enter weight: ")))
    profits.append(int(input("Enter profit: ")))

capacity = int(input("Enter knapsack capacity: "))

dp = [[0] * (capacity + 1) for i in range(n + 1)]

for i in range(1, n + 1):
    for w in range(capacity + 1):
        if weights[i - 1] <= w:
            dp[i][w] = max(
                profits[i - 1] + dp[i - 1][w - weights[i - 1]],
                dp[i - 1][w]
            )
        else:
            dp[i][w] = dp[i - 1][w]

print("Maximum Profit:", dp[n][capacity])
```

DAA V

```
def solve(board, row, n):
    if row == n:
        return True

    for col in range(n):
        safe = True

        for i in range(row):
            if board[i][col] == 1:
```

```
                safe = False
                break
            if abs(i - row) == abs(col - (board[i].index(1))):
                safe = False
                break

        if safe:
            board[row][col] = 1

            if solve(board, row + 1, n):
                return True

            board[row][col] = 0

    return False


n = int(input("Enter N: "))
board = [[0] * n for i in range(n)]

board[0][0] = 1

if solve(board, 1, n):
    for row in board:
        print(row)
else:
    print("Solution does not exist")
```